

Tutorial 2: Rotated Surface Codes

From local checks to lattice surgery

How can we protect quantum information when a device can only couple nearby qubits?

In this tutorial, we arrange stabilizer checks on a lattice, use their geometry to understand errors, and then turn a quantum memory into a logical CNOT. We assume familiarity with Pauli operators, stabilizers, CNOT gates, and the Pauli frame, as introduced in the Shor-code tutorial.

1 From Shor's code to local checks

A **geometrically local code** places qubits in space and uses checks whose support stays within a bounded neighborhood as the code grows. The surface code is a two-dimensional example. Its geometry also makes it a **topological code**: local checks cannot distinguish the logical states, while certain extended strings act on the encoded information.

The connection to Shor's code is the stabilizer condition:

$$S_i|\psi_L\rangle = |\psi_L\rangle.$$

Both codes use separate X - and Z -type checks, a structure called **CSS**. We still infer errors from check outcomes without measuring the encoded amplitudes. What changes is the arrangement and support of the checks. The nine-qubit Shor code and the distance-three rotated surface code both have parameters $[[9, 1, 3]]$, but they have different stabilizer groups.

Why use this geometry?

Each check couples only a few neighboring qubits. Increasing the patch size increases the minimum size of an undetectable logical Pauli operator, called the **code distance** d . This provides a route to stronger protection without requiring each qubit to connect to more distant neighbors.

Surface codes are used in experiments: repeated error correction and decreasing logical error with increasing distance have been demonstrated under suitable operating conditions [4]. They are not automatically beneficial at every physical error rate. Performance depends on the noise, measurement circuit, and decoder.

The cost is substantial. A square rotated patch uses d^2 data qubits for one logical qubit, plus measurement ancillas. Checks must be repeated, their results decoded, and logical gates require additional space and time. Locality is useful, but it does not make fault tolerance free.

2 Why is it called rotated?

In the conventional surface-code lattice, data qubits sit on **edges**. An X check acts on edges meeting at a vertex; a Z check acts around a face. A finite patch has truncated checks at its boundaries.

The rotated construction chooses boundaries at 45° to this underlying lattice and removes surplus boundary qubits. After redrawing, the data qubits form a square grid and sit at the **corners of checkerboard faces**. This changes the patch cut; merely turning a drawing does not change the code [1].

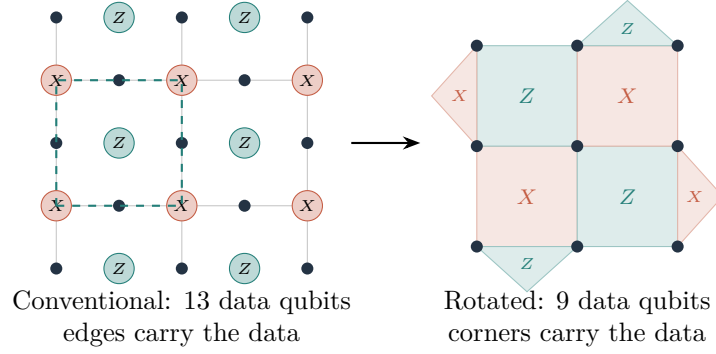


Figure 1: Two distance-three patches. Black dots are data qubits; colored markers or faces are checks. The dashed square highlights one Z plaquette in the conventional lattice. The right drawing places data at face corners. Ancillas are excluded from the counts.

For the usual square patches with the same distance,

$$n_{\text{conventional}} = d^2 + (d - 1)^2, \quad n_{\text{rotated}} = d^2.$$

Thus $d = 3$ requires 13 or 9 data qubits, respectively. Both patches encode one logical qubit. Rotation saves space while preserving distance; it does not by itself establish a better error rate under every noise model.

Our drawing convention

Throughout the tutorial, X checks are orange and Z checks are teal. A black dot is a data qubit. A check region specifies which data qubits participate; a measurement ancilla, when shown, sits near its center.

We label a boundary by the *logical operator supported along it*: the top and bottom are X boundaries, and the left and right are Z boundaries. This is a patch-operator convention, following Litinski’s visual representation [2]. It does not name the truncated check at that edge.

3 Reading the checks and boundaries

For a face f , the corresponding stabilizer is

$$S_X(f) = \prod_{j \in \text{corners}(f)} X_j, \quad S_Z(f) = \prod_{j \in \text{corners}(f)} Z_j.$$

Bulk checks have weight four; the boundary half-faces have weight two. In Figure 2, qubit $q_{x,y}$ has label $5y + x$.

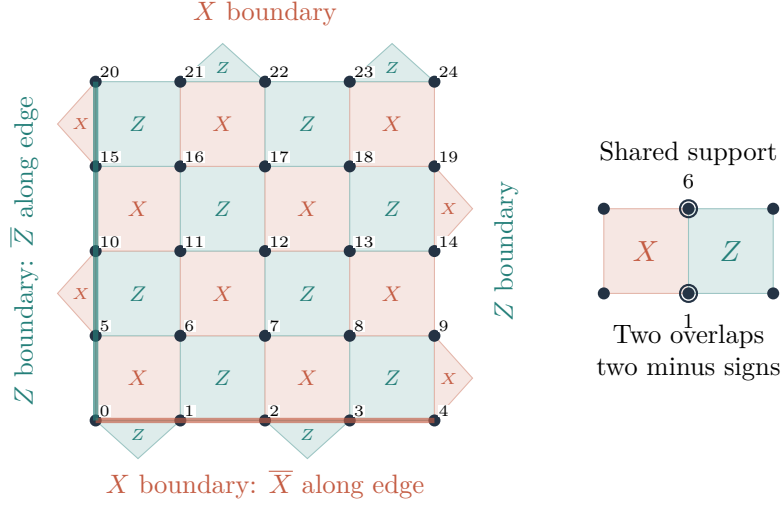


Figure 2: A distance-five rotated patch: 25 data qubits and 24 independent checks. Letters label check types, not extra data qubits. The paths represent $\bar{X}$ and $\bar{Z}$. The inset isolates the two shared data qubits of the neighboring checks used below.

For example, neighboring bulk checks are

$$S_X = X_0 X_1 X_5 X_6, \quad S_Z = Z_1 Z_2 Z_6 Z_7.$$

They overlap at qubits 1 and 6. Each overlap contributes a minus sign when we exchange X and Z , so

$$S_X S_Z = (-1)^2 S_Z S_X = S_Z S_X.$$

Every unlike pair of checks shares zero or two qubits; like-type checks commute automatically. The checks can therefore constrain the state simultaneously.

In contrast, a single Z_6 anticommutes with the displayed X check because it overlaps once. Its outcome changes from $+1$ to -1 . We record $+1$ as syndrome bit 0 and -1 as bit 1.

4 Does the measurement order matter?

The stabilizers commute, so the order of *ideal projective check measurements* does not matter. Their physical implementation is a different question.

To measure a Z check, prepare an ancilla in $|0\rangle$, apply CNOTs from each participating data qubit to the ancilla, and measure the ancilla in Z . For an X check, prepare $|+\rangle$, use the ancilla as the CNOT control, and measure it in X .

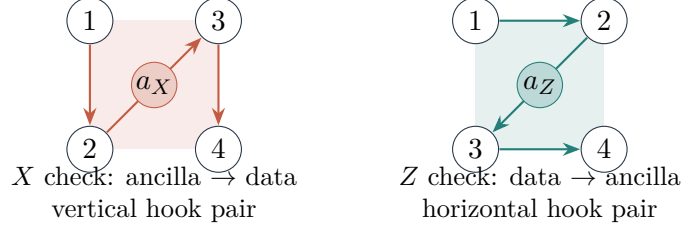


Figure 3: One compatible four-layer bulk schedule for our patch orientation. Numbers give the CNOT interaction layer at each corner; the paths are ordering guides, not data-data gates. Boundary checks omit missing corners while retaining the layer numbers.

For this orientation, we use an N-shaped order for X checks and a Z-shaped order for Z checks:

$$\begin{aligned} X &: \text{NW} \rightarrow \text{SW} \rightarrow \text{NE} \rightarrow \text{SE}, \\ Z &: \text{NW} \rightarrow \text{NE} \rightarrow \text{SW} \rightarrow \text{SE}. \end{aligned}$$

These are four shared time layers across the patch. The orders must avoid two gates using the same qubit at once and implement compatible parity measurements; independently choosing an order for every face is not sufficient.

Why N and Z?

A fault on a measurement ancilla can propagate to several data qubits. For example, an X fault on the control ancilla of an X check, after its second CNOT, spreads to its last two targets. The resulting correlated pair is called a **hook error**.

Here the last two data qubits touched by an X check form a vertical pair, transverse to our horizontal $\bar{X}$. For a Z check, they form a horizontal pair, transverse to our vertical $\bar{Z}$. This prevents these two-qubit hooks from taking a two-site step along the corresponding shortest logical string. Good schedules account for this orientation [5].

The letters are drawing mnemonics, not universal rules: rotating the patch or exchanging X and Z changes the appropriate order. A full circuit still needs fault analysis, including preparation, readout, and boundary operations.

5 Errors, strings, and Pauli charges

An error is a Pauli operator on data qubits; a syndrome is the set of check outcomes it changes. These are different objects. A line in the following pictures represents a *product* of errors along a path, not a particle’s measured trajectory.

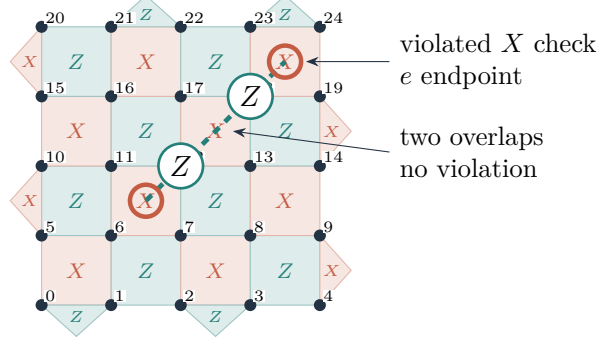


Figure 4: The error $E = Z_{12}Z_{18}$ flips the two circled X checks. The intermediate X check overlaps both error factors, so its outcome is unchanged. The dashed line joins check centers through the affected data qubits.

We call a violated check an **endpoint charge**. In the usual anyon language, a violated X check is an e charge, created at an endpoint of a Z string; a violated Z check is an m charge, created by an X string [6]. These are not electric charges. Here, “Pauli charge” refers to this syndrome content of a Pauli string.

Two identical endpoint charges cancel when strings join, because syndromes add modulo two:

$$\mathbf{s}(E_1 E_2) = \mathbf{s}(E_1) \oplus \mathbf{s}(E_2).$$

A $Y = iXZ$ error combines both kinds of syndrome. In the bulk, a string has two endpoints. At a suitable boundary, one endpoint can disappear without violating a check.

Multiplying an error by a stabilizer changes its physical support without changing its syndrome or action on encoded states. This is why a decoder chooses a *recovery class*, rather than reconstructing a unique history. A recovery C succeeds if $CE \in \mathcal{S}$, up to global phase. Having zero residual syndrome alone is not enough: CE could still be a logical operator.

6 Zooming out to a logical patch

We can now hide the microscopic checks and draw one square per logical qubit. This patch view is inspired by *A Game of Surface Codes* [2]; the figures here are original schematics.

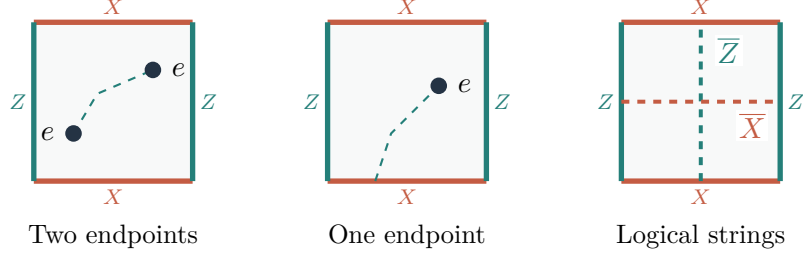


Figure 5: Left: a Z string with two detectable e endpoints. Middle: an endpoint is absorbed at an X boundary. Right: $\overline{Z}$ connects the two X boundaries, while $\overline{X}$ connects the two Z boundaries. Neither logical string has a syndrome endpoint.

The boundary convention is worth making explicit:

Boundary label	Operator along the edge	String that can end there
X (top/bottom)	$\overline{X}$	Z string (e charge)
Z (left/right)	$\overline{Z}$	X string (m charge)

With our charge convention, the X boundaries absorb e and are called rough; the Z boundaries absorb m and are called smooth. We use explicit operator and endpoint labels because boundary naming conventions vary between references.

Seeing the logical algebra

For our numbered patch, choose

$$\overline{X} = X_0 X_1 X_2 X_3 X_4, \quad \overline{Z} = Z_0 Z_5 Z_{10} Z_{15} Z_{20}.$$

Both commute with every check, but overlap each other on exactly one qubit:

$$\overline{X}\overline{Z} = -\overline{Z}\overline{X}.$$

The crossing in the square is the geometric version of this odd overlap. A shortest nontrivial string contains d data qubits. Contractible closed strings are stabilizers; a string spanning the appropriate opposite boundaries can act on the logical qubit without announcing itself in the syndrome.

The same picture explains decoding failure: the actual error and chosen recovery can join into a spanning logical string even though their endpoints match and cancel.

7 A memory experiment with repeated checks

How reliably does the patch preserve a logical qubit for T rounds?

1. **Prepare.** For a Z -basis memory, initialize data in $|0\rangle^{\otimes d^2}$ and establish the checks, recording the initially random X -check signs in the frame. For an X -basis memory, start with $|+\rangle^{\otimes d^2}$ and exchange X and Z .
2. **Repeat.** Measure checks with ancillas for T rounds. A specified circuit noise model can include data, gate, reset, idle, and measurement faults.
3. **Decode.** Use the measurement history to infer a recovery and track its Pauli frame.
4. **Read out.** Measure all data in the chosen basis. Their parities give the final logical measurement and final compatible check values. Include these in decoding before adjusting the logical result.

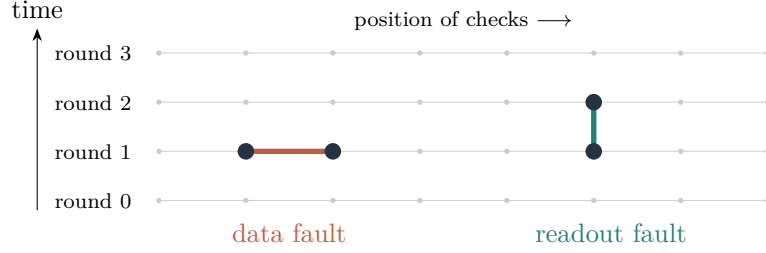


Figure 6: A slice of a space-time detection graph for one check type. A data fault can join neighboring checks in space. A flipped check readout can join events on the same check in consecutive rounds. Real circuit faults can also create diagonal or more complicated event patterns.

Let $s_i(t)$ be a check's measured bit in round t . Away from initialization and readout, a **detection event** is

$$D_i(t) = s_i(t) \oplus s_i(t-1).$$

A persistent data error changes the syndrome when it occurs; it does not create a new event every round. An isolated wrong readout usually creates events in two neighboring rounds. Initial comparisons use only known preparation constraints; final comparisons use the checks inferable from the data-measurement basis.

For N shots at fixed d and T , report

$$\hat{p}_{L,B}(T) = \frac{\text{incorrect frame-adjusted logical readouts in basis } B}{N}, \quad B \in \{X, Z\}.$$

These are failure probabilities for the entire experiment, not automatically per-round rates. Run both bases: a Z readout detects logical X/Y errors, while an X readout detects logical Z/Y errors. Numerical performance requires a specified noise model, schedule, and decoder.

8 Lattice surgery: measuring between patches

A memory preserves one logical qubit. To compute, we also need operations that couple logical qubits. Lattice surgery implements joint logical measurements while retaining local physical interactions [1].

Suppose two patches share facing Z boundaries. We temporarily replace checks near the interface with compatible checks that join the patches. Products of their outcomes, together with known check and frame signs, reveal $\bar{Z}_1\bar{Z}_2$. Restoring separate patches completes a parity-measurement operation. Facing X boundaries similarly allow $\bar{X}_1\bar{X}_2$.

The measured eigenvalue gives a *relation* between logical values, not either value separately. For example,

$$\Pi_{ZZ}^{(+)}|+\rangle_L|+\rangle_L \propto |00\rangle_L + |11\rangle_L, \quad \Pi_{ZZ}^{(+)} = \frac{I + \bar{Z}_1\bar{Z}_2}{2}.$$

The -1 branch instead gives an odd-parity Bell state. The outcome is recorded and its effect tracked.

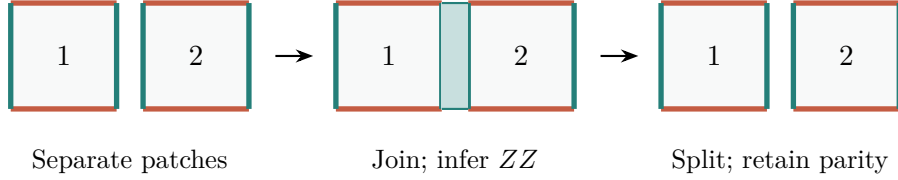


Figure 7: A patch-level ZZ measurement. The teal seam joins facing Z edges. “Join” means changing and repeatedly measuring interface checks, not directly measuring all data qubits along the seam. The picture omits physical bridge-qubit and gate details.

Where does the time cost enter?

One noisy interface measurement is insufficient. In standard distance- d protocols, a surgery stage uses on the order of d syndrome rounds, together with suitable spatial separation, to protect the parity result against measurement faults. Splitting also needs correct frame bookkeeping; separating drawings alone is not a physical protocol.

Lattice surgery is useful when joint logical measurements and entangling gates must respect local connectivity. It is one route to logical computation, not the only one. We will use two such parity measurements to build a CNOT.

9 A CNOT from parity measurements

Let C and T be arbitrary control and target logical qubits. Prepare an ancillary *logical patch* A in $|+\rangle_L$; this is different from a single physical check ancilla.

Use the following sequence, where outcome bit m denotes eigenvalue $(-1)^m$. In a noisy implementation, a, b, c are decoded logical outcomes, interpreted using the incoming Pauli frame:

Step	Operation	Outcome
1	Measure $\bar{Z}_C \bar{Z}_A$	a
2	Measure $\bar{X}_A \bar{X}_T$	b
3	Measure $\bar{Z}_A$ and discard A	c
4	Track $\bar{Z}_C^b \bar{X}_T^{a \oplus c}$	Pauli frame update

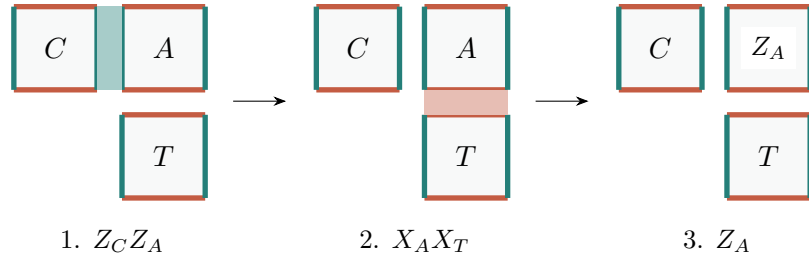


Figure 8: An L-shaped layout allows C – A to meet on Z edges and A – T on X edges. Highlighted seams show successive parity measurements; each stage restores the separate patches before the next. The last stage reads out the ancilla patch.

Why does this give CNOT?

For logical computational-basis input $|x\rangle_C |y\rangle_T$, the first measurement fixes the ancilla value to $x \oplus a$. The second creates the two branches related by $X_A X_T$. The final result c selects the target value $y \oplus x \oplus a \oplus c$, with relative phase $(-1)^{bx}$, up to an outcome-dependent global phase.

Thus the uncorrected branch acts as

$$|\psi\rangle_{CT} \mapsto \bar{Z}_C^b \bar{X}_T^{a \oplus c} \text{CNOT}_{C \rightarrow T} |\psi\rangle_{CT},$$

up to normalization and global phase. The frame update removes those known byproducts in subsequent interpretation. Linearity makes the argument valid for superpositions and entangled inputs, not just basis states.

Unlike memory checks, these two joint observables anticommute on A . Their *sequence* is part of the logical protocol and must not be freely reordered.

10 The same operation as a ZX graph

ZX diagrams describe logical linear maps. Wires are logical systems, and colored nodes, called **spiders**, represent copying or matching amplitudes in a chosen basis. They are not physical stabilizer faces or endpoint charges.

For zero phase, a Z spider (teal/green) with m inputs and n outputs is

$$Z_{m,n} = |0\rangle^{\otimes n} \langle 0|^{\otimes m} + |1\rangle^{\otimes n} \langle 1|^{\otimes m}.$$

An X spider (orange/red) uses $|+\rangle, |-\rangle$ instead. Normalization scalars are often suppressed when drawing a fixed measurement branch. Connected spiders of the same color fuse; a split and merge in the corresponding basis therefore have a compact graphical description [3].

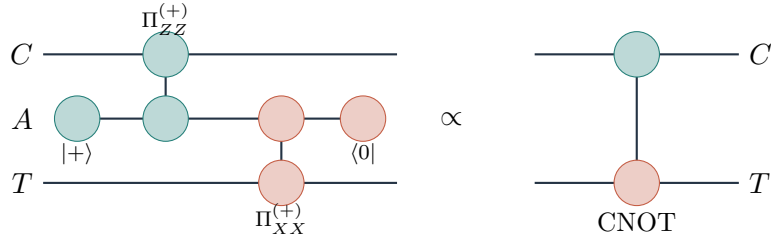


Figure 9: The all-+1 branch of the parity-measurement CNOT, reduced to its ZX form. Time runs left to right. A ZZ projector connects two Z spiders; an XX projector connects two X spiders. The one-legged Z preparation is $|+\rangle$ and the one-legged X effect is $\langle 0|$, up to scalars.

In the left graph, fuse the connected Z spiders, including the $|+\rangle_A$ preparation. Fuse the connected X spiders, including the $\langle 0|_A$ effect. The remaining graph is a Z spider on the control wire joined to an X spider on the target wire: the standard CNOT graph, up to the suppressed scalar.

The other outcome branches have Pauli byproducts. For our measurement convention, these are precisely $\overline{Z}_C \overline{X}_T^{a \oplus c}$ from the previous section. A positive-branch graph alone does not describe the deterministic protocol; the outcome record and frame update complete it.

What happened to the charges?

At the physical level, endpoint charges tell us which local checks an error violates. At the logical level, Pauli propagation tells us how a pending frame moves through the operation:

$$\begin{aligned} X_C &\mapsto X_C X_T, & Z_C &\mapsto Z_C, \\ X_T &\mapsto X_T, & Z_T &\mapsto Z_C Z_T. \end{aligned}$$

Here X_C, Z_C, X_T, Z_T denote logical operators. This connects the syndrome-and-frame story to the logical graph without identifying microscopic charges with spiders. The patch diagrams explain *where* the operation can happen; the ZX graph explains *which logical map* it implements.

References

- [1] C. Horsman, A. G. Fowler, S. Devitt, and R. Van Meter, *Surface code quantum computing by lattice surgery*, New J. Phys. 14, 123011 (2012). [arXiv:1111.4022](#).
- [2] D. Litinski, *A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery*, Quantum 3, 128 (2019). [doi:10.22331/q-2019-03-05-128](#). Patch notation and Appendix A motivate the change of visual scale used here.
- [3] N. de Beaudrap and D. Horsman, *The ZX calculus is a language for surface code lattice surgery*, Quantum 4, 218 (2020). [arXiv:1704.08670](#).
- [4] Google Quantum AI and Collaborators, *Quantum error correction below the surface code threshold*, Nature 638, 920–926 (2025). [doi:10.1038/s41586-024-08449-y](#).
- [5] NVIDIA, *CUDA-QX: QEC Codes*, surface-code stabilizer scheduling documentation. [Online documentation](#), accessed 17 September 2026.
- [6] A. Kitaev, *Fault-tolerant quantum computation by anyons*, Annals of Physics 303, 2–30 (2003). [arXiv:quant-ph/9707021](#).